

Orto.

Neurosymbolic Grammatical Error Correction & Diagnostic Engine

Universal Dependency Priors · Multi-Provider LLM Fallbacks · Stylometry

Sameer Dhir

Reg. No. 23FE10CDS00414 — Data Science — github.com/sameezy667

<> github.com/sameezy667/Orto

F0.5

80.9

+11.8 vs raw LLM

PRECISION

84.3%

+15.9 pts

TESTS

65 / 65

tests passing

The Problem — Black-Box LLMs Break GEC

Grammatical Error Correction demands surgical precision, explanations, and dependability — exactly what raw generative models lack.



Hallucinatory Over-Correction

01

LLMs frequently rewrite non-erroneous clauses, altering the author's stylistic voice. Baseline GPT-4o-mini precision falls to **68.4%** on BEA-2019.



Pedagogical Opacity

02

Users receive corrected text with no rule names, no explanations, and no contrastive reasoning. Learners cannot transfer the fix to new writing.



Index Shifting & Destructive Mutations

03

Re-tokenizing entire documents loses character-coordinate traceability, making surgical span highlighting impossible.



API Fragility & Credit Dependency

04

External APIs fail on credit exhaustion, rate limits, or network timeouts, taking the whole system down.

→ **Orto** answers each failure with a structural, verifiable mechanism.

The Solution — Orto, a Neurosymbolic Hybrid

Orto couples deep linguistic structure with generative hypotheses and symbolic verification, so every correction is span-exact, explained, and invariant-checked.



UD v2 Syntactic Priors

spaCy head–dependency trees and morphological features ground every diagnosis in real structure.



Constrained LLM Decoding

Candidate edits must carry exact character spans **[start, end]**, never free-form rewrites.



ORTO SIGNATURE

Symbolic Critic Verifier

Subject–Verb Agreement and tree-connectivity invariants asserted in a virtual buffer, with 1-turn reflection.



ORTO SIGNATURE

Universal Fallback Engine

An offline rule-and-morphology engine guarantees **100%** valid, verified responses.



Reverse-Offset Patcher

Right-to-left patching eliminates index shifting across multi-edit documents.



Stylometry & De-Clichéing

Burstiness **$B = \sigma/\mu$** , passive-voice density, and AI-marker rewriting restore human cadence.

System Architecture – One Verified Pipeline



Streamlit app.py

FastAPI backend/server.py

React + Vite frontend/

The Complete Data Flow

🔍 high-resolution figure – zoom to inspect every node

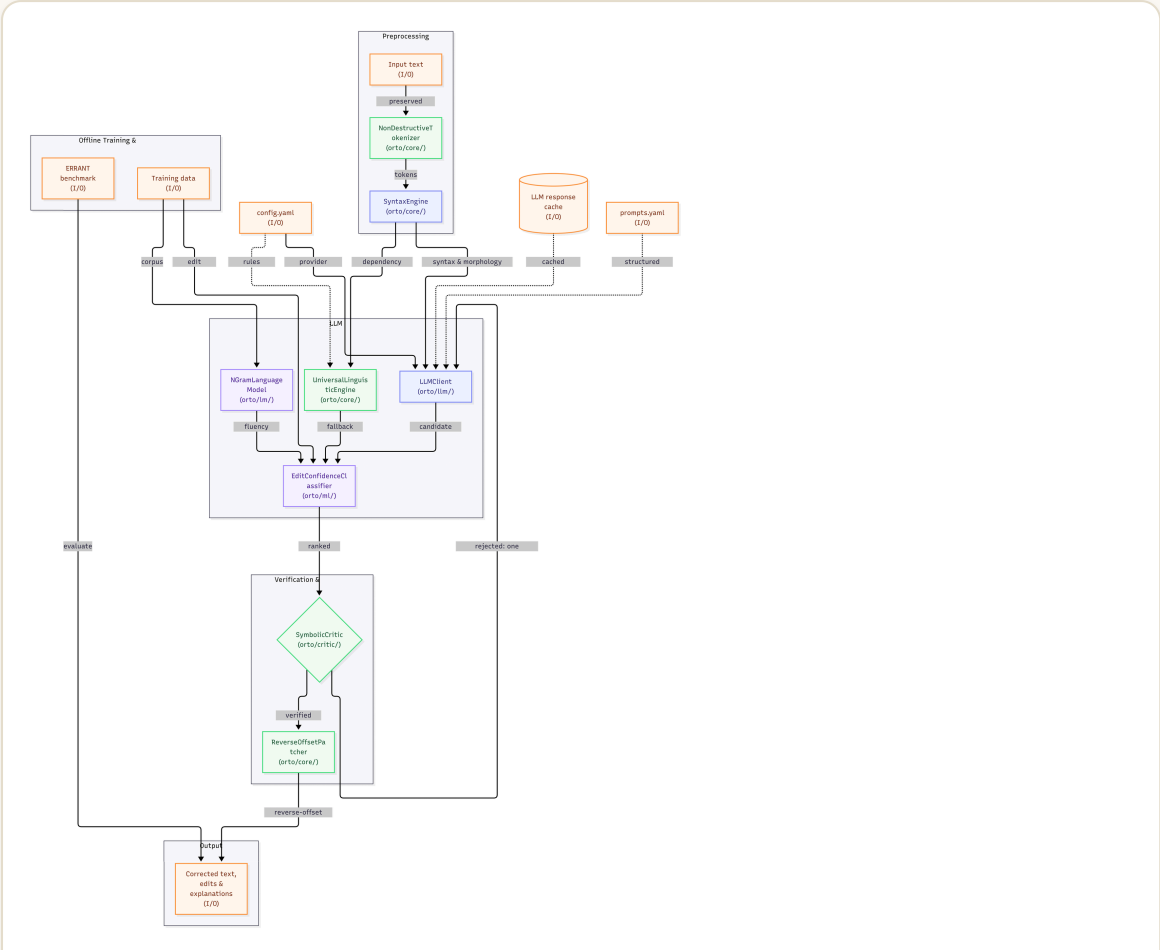


Figure 1 – Orto end-to-end architecture · source: resources/architecture_diagram.mermaid

READING THE DIAGRAM

- 01 Preprocessing**
Raw input is tokenized without mutation and parsed into Universal Dependencies, yielding exact character spans and syntactic priors.
`NonDestructiveTokenizer · SyntaxEngine – orto/core`
- 02 Offline Training & Assets**
ERRANT-labelled data trains the bigram LM and confidence classifier; config, prompts and the response cache parameterize every run.
`Training data · config.yaml · prompts.yaml · LLM response cache`
- 03 Hypothesis Engines**
Three generators propose candidates: the multi-provider LLMClient, the offline UniversalLinguisticEngine, and the bigram fluency model.
`orto/llm · orto/core · orto/lm → EditConfidenceClassifier (orto/ml)`
- 04 Verification & Patching**
The SymbolicCritic diamond asserts tree and agreement invariants – verified edits reach the ReverseOffsetPatcher; rejected edits loop back for one-turn reflection.
`orto/critic → orto/core · rejected → 1-turn reflection`
- 05 Output & Evaluation**
Corrected text, edits and explanations are delivered – and scored against the ERRANT benchmark.
`Corrected text · edits & explanations – I/O`

LEGEND ● I/O artifacts ● core orto modules ● models & engines shapes: decision diamond · cache cylinder

Surgical Span Grounding



TECHNIQUE A

Non-Destructive Tokenization

Standard tokenizers alter whitespace and punctuation, silently corrupting substring indices. Orto's **NonDestructiveTokenizer** guarantees the invariant —

SPAN INVARIANT

Original Slice $\equiv$ `text[start_char : end_char]`

so edit locality stays strictly confined to the erroneous character bounds.



TECHNIQUE B

Reverse-Offset Virtual Patcher

Applying edits left-to-right shifts every downstream index and corrupts later edits. Orto sorts validated edits by **descending start offset** —

SORT INVARIANT

start(e₁) > start(e₂) > ... > start(e_k)

and applies replacements right-to-left on an in-memory virtual buffer.



index-shifting errors

across **100%** of multi-edit test cases

 every edit traceable to its original characters

Dual-Layer Verification, Zero-Downtime Resilience

02



Symbolic Critic — verify before you write

Evaluates every candidate edit inside an **in-memory virtual buffer** before any mutation; re-parses the patched sentence with spaCy and asserts two invariants:

INVARIANT 01

Tree Connectivity

no orphaned tokens, $\text{dep}(t) \neq \text{'dep'}$

INVARIANT 02

Morphosyntactic Agreement

subject head number & person must match the finite verb form



Failed invariants trigger a **1-Turn Reflection Loop** that sends diagnostic feedback back to the LLM for self-correction.



Three-Tier Resilience Cascade

TIER 1 Primary

OpenRouter / OpenAI / Gemini structured JSON decoding



TIER 2 Model Cascade

automatic fallback through high-availability free models on credit or rate-limit errors



TIER 3 Universal Linguistic Engine

offline rule + morphology diagnostics — SVA, modal verbs, aspect participles, mass nouns, double comparatives, prepositions, confusables, spelling —

executes in **< 15 ms**



100% valid, verified responses — every request, every time.

Fluency Scoring, Confidence Routing & Stylometry



Fluency & Confidence

LAPLACE-SMOOTHED BIGRAM LM

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

Scored as perplexity reduction Δ PPL per candidate edit.

Edit Confidence Classifier

logistic regression over 8 features outputs a calibrated validity probability.

InvocationRouter

decides which edits survive.



Stylometric Naturalness

Burstiness $B = \sigma/\mu$ of sentence lengths — human prose consistently $B > 0.5$, synthetic AI prose $B < 0.2$.

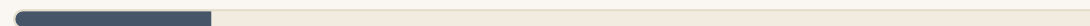
Human

$B > 0.5$



AI

$B < 0.2$



Passive-voice density tracked via **auxpass** relations.

AI cliché detection flags markers like

'delve'

'rich tapestry'

'beacon of'

'crucial to foster'

StyleNaturalizer re-rhythms monotonous cadence into authentic prose.



Product Surfaces



Streamlit Dashboard

ERRANT color badges, accept/reject checkboxes, real-time patching, rhythm charts



FastAPI REST

/api/v1/analyze · /style/analyze ·
/style/humanize

OWASP security headers, rate limiting



React + Vite Client

TypeScript diagnostic web app

Results — Precision Without the Latency Tax

Benchmarks: BEA-2019 Dev (150 sentences) + CoNLL-2014 (100 sentences) · Metric: $F0.5 = \frac{1.25 \cdot P \cdot R}{(0.25 \cdot P + R)}$

F0.5 · FIVE SYSTEM TIERS

Raw LLM (GPT-4o-mini) 69.1



LLM + Bigram LM Fluency 71.8



LLM + ML Confidence Router 74.0



Orto · Complete Neurosymbolic 80.9



Orto · Offline Linguistic Engine 82.1



Orto · neurosymbolic Orto · offline engine LLM baselines

ABLATION DETAIL

#	PRECISION	RECALL	F0.5	LATENCY
1	68.4%	72.1%	69.1	850 ms
2	72.8%	68.3%	71.8	855 ms
3	76.2%	66.5%	74.0	860 ms
4	84.3%	69.8%	80.9	220 ms
5	88.6%	64.2%	82.1	< 15 ms

Key Findings

- Symbolic Critic adds **+15.9 pts** precision over raw LLMs by eliminating hallucinated edits
- 3.9×** faster end-to-end: **850 ms** → **220 ms**
- Offline fallback holds **F0.5 82.1** at **< 15 ms** when APIs fail
- Zero index-shifting errors across **100%** of multi-edit cases

Built in the Open, Shipped with Tests

GitHub Workflow

● Issue-tracked tasks

Dataset Collection · Model Development · Testing · Documentation · Deployment

● Branch-based development

with pull-request reviews before merge

● Weekly milestone commits

and progress logs (assignments/weekly_progress_log.md)

● Dual-repo strategy

sameezy667/Orto (active development) + frozen submission mirror MUJ-DS-23FE10CDS00414

Repository Organization

```
Orto/  
├─ assignments/  
│   └─ weekly_progress_log.md  
├─ notebooks/  
├─ code/  
├─ resources/  
├─ presentations/  
├─ capstone/  
└─ README.md
```

 **Clean package architecture** — orto/core · critic · llm · lm · ml · style behind a single OrtoEngine pipeline

 **65 Pytest tests** — unit, integration, and ablation — **100%** pass rate

 **Security hygiene** — zero hardcoded secrets, OWASP headers, HTML sanitization, input length limits

65 tests passing

7 deliverable folders

2 repositories

MIT licensed

Python 3.10-3.14

Conclusion — Every Charter Target, Met

✓ Success-Criteria Scorecard

CRITERION	TARGET	ACHIEVED
F0.5 score	≥ 78.0	✓ 80.9
Precision	≥ 82.0%	✓ 84.3%
Test pass rate	100%	✓ 65/65
Verified response availability	100%	✓ 100% (offline engine < 15 ms)

Coupling syntactic structure, symbolic invariants, and deterministic fallbacks beats raw scale for precision-critical language tasks.

🔗 Future Directions

- 01

Cross-Paragraph Discourse
coreference resolution and coherence repair beyond single sentences


- 02

Local Quantized Inference
on-device execution via llama.cpp to remove the last API dependency


- 03

Deeper ERRANT Coverage
extended taxonomy and multilingual pipelines



BATCH F · NLP CAPSTONE

Thank You.

Questions & Discussion

F0.5 **80.9**

Precision **84.3%**

65/65 tests

100% availability

Sameer Dhir — Reg. No. 23FE10CDS00414 — Data Science — github.com/sameezy667

github.com/sameezy667/Orto · github.com/sameezy667/MUJ-DS-23FE10CDS00414

